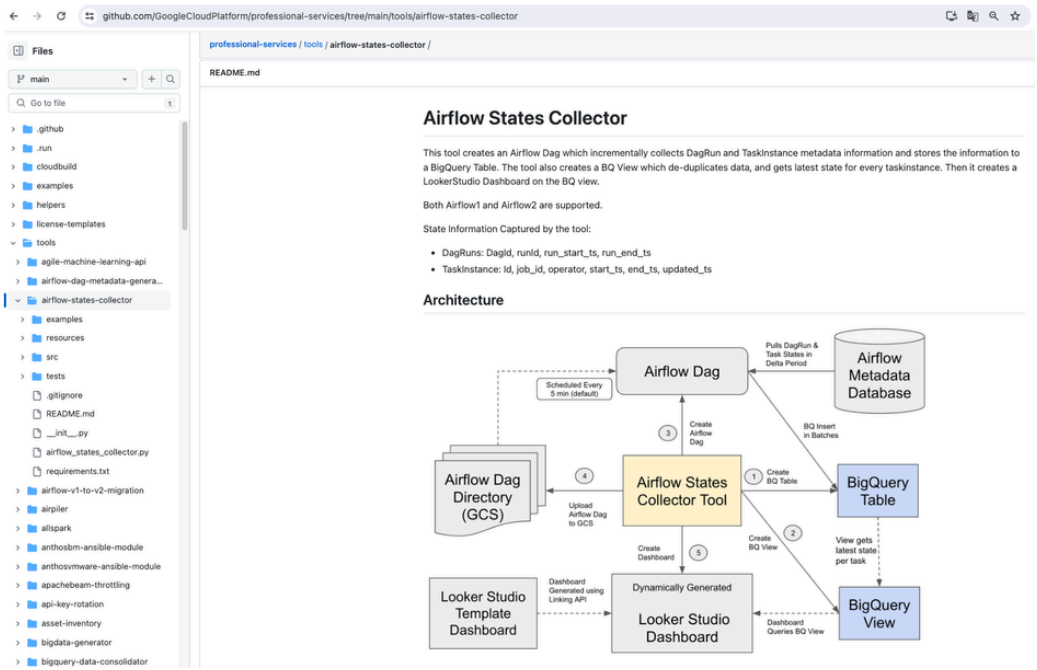
Creating Detailed Logs in Airflow (Airflow States Collector)

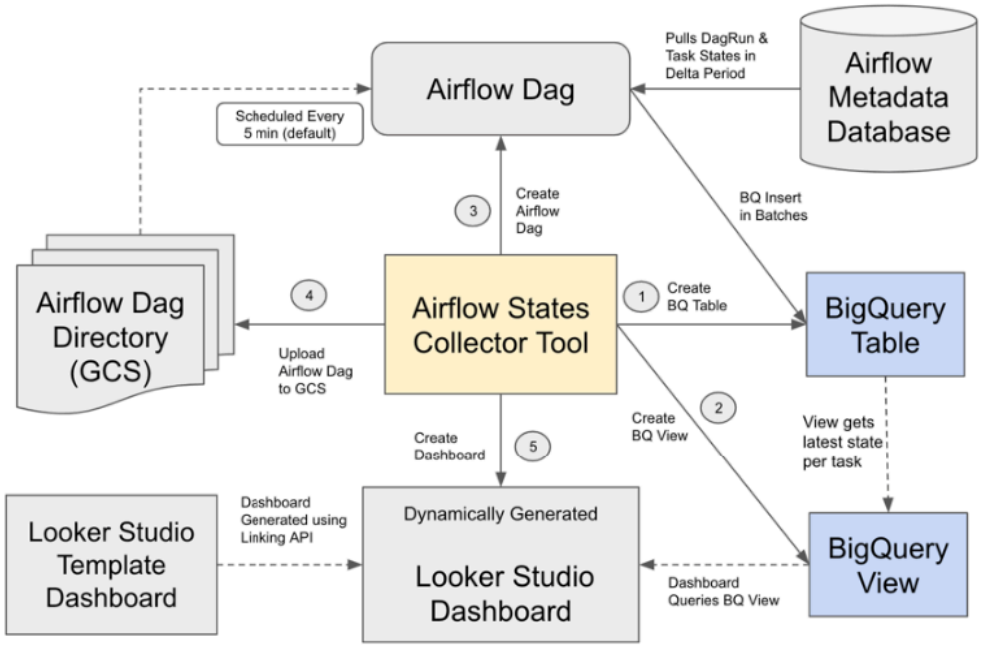
Maintaining logs for the Airflow system and taking necessary actions based on the observed logs is a crucial process in workflows. Typically, logging is attempted by developing code using Airflow libraries, which can be both administratively challenging and labor-intensive. For this task, you can use the Airflow States Collector in your projects. It meets general needs with its parameters and provides easy management with ready-made dashboards. Since these dashboards are prepared with Looker Studio, there is no additional cost. Visually, it makes monitoring this information much more elegant and comprehensible. You can quickly and easily implement this setup in your projects, simplifying the Airflow monitoring process.

Source: Airflow-states-collector_github_link

Clicking the link opens the GitHub project page. Here, you will find step-by-step instructions on how to implement it and example screenshots.



Architecture



First, the necessary tables and views are created in BigQuery. Then, using the 'airflow_states_collector.py' file, a DAG is set up in Airflow to run every 5 minutes, collecting logs and transferring them to BigQuery. Finally, dashboards are created using this information, and as data comes in, they are updated to visualize detailed information about the Airflow logs.

Implementation

If not already installed, the necessary installations should be performed.

- pip: pip documentation v24.2
- virtualenv: virtualenv
- Google Cloud SDK: Cloud SDK - Libraries and Command Line Tools | Google Cloud

First, clone the GitHub repository using git clone:

```
$ git clone https://github.com/GoogleCloudPlatform/professional-services.git
```

Navigate to the relevant directory with the cd command:

```
$ cd professional-services/tools/airflow-states-collector
```

Create a virtual environment:

```
$ virtualenv --python python3 env
$ source env/bin/activate
```

Install the required packages (BigQuery, Jinja2, etc.):

```
$ pip install -r requirements.txt
```

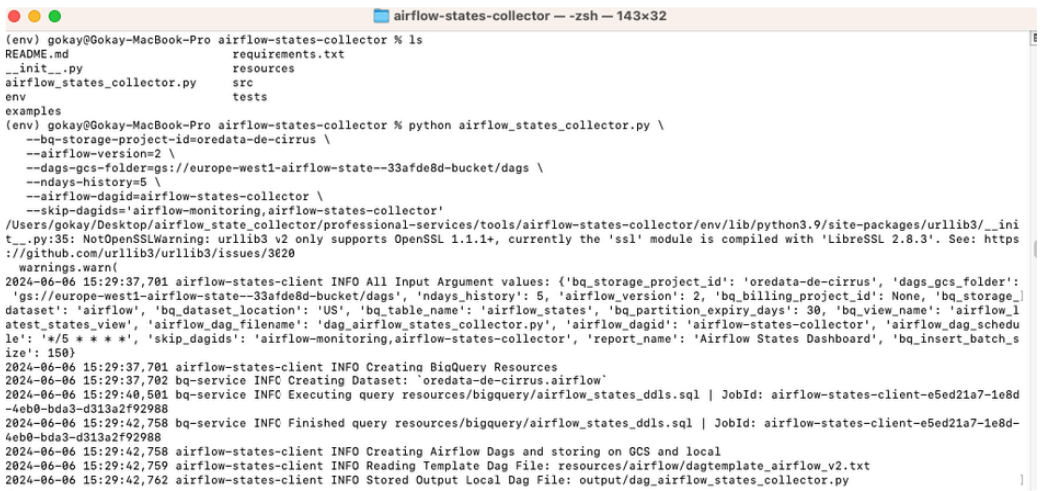
Connect to the Google environment and select the project:

```
$ gcloud init
```

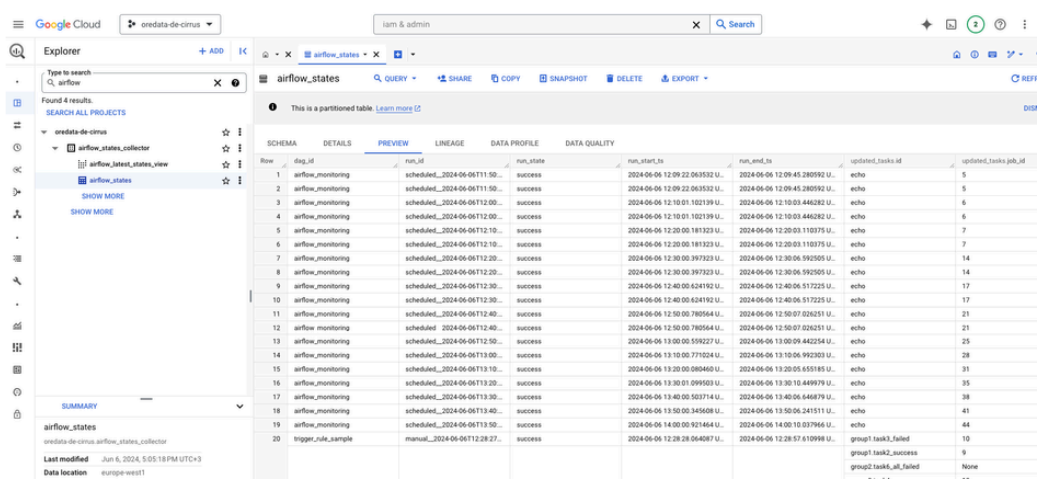
Make the necessary changes in the code and run it. The history is set to 5 days; if you encounter a query size error when the DAG runs, you may need to reduce this further. Additionally, it is recommended that the BigQuery dataset location matches the composer location.

```
python airflow_states_collector.py \
   --bq-storage-project-id=oredata-de-cirrus \
   --airflow-version=2 \
   --dags-gcs-folder=gs://europe-west1-airflow-state--33afde8d-bucket/dags \
   --ndays-history=5 \
   --airflow-dagid=airflow-states-collector \
   --skip-dagids='airflow_monitoring,airflow-states-collector'\
   --bq-dataset-location='europe-west1'\
   --bq-storage-dataset='airflow_states_collector'
```

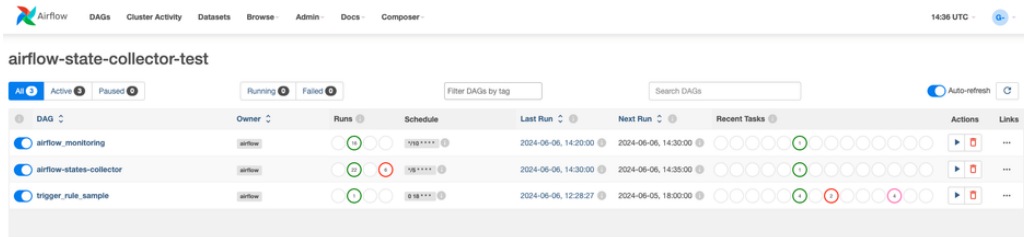
The output is as follows:



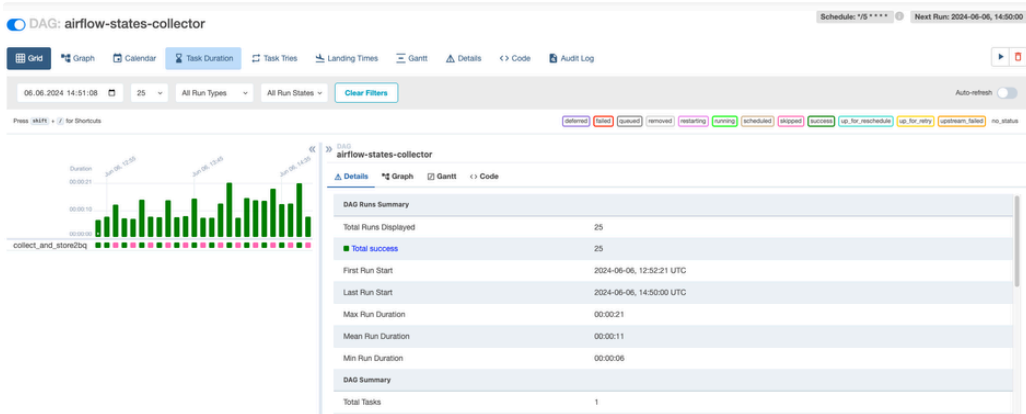
When the execution is completed, the necessary dataset and materials will be created in BigQuery.



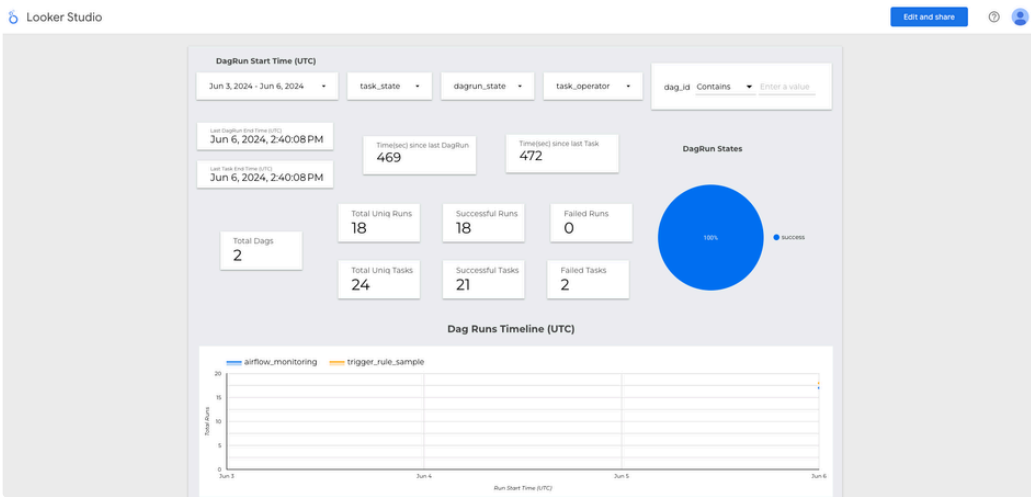
The DAG is displayed on the Airflow screen.



When it runs successfully and completes without any changes, or if it is skipped, it will be displayed.



Dashboards are viewed with Looker Studio. It allows filtering and viewing with desired details within the specified range.



It allows performing necessary inspections on DAG breakdowns and uniquely, providing minimum, median, and maximum durations for the DAG.

DAG Statistics

	Dag Id	Total Runs	Successful Runs	Failed Runs	Min Run Duration (sec)	Median Run Duration (sec)	Max Run Duration (sec)	Min Run Start Time (UTC)	Max Run Start Time (UTC)
1.	trigger_rule_sample	1	1	0	29	29	29	Jun 6, 2024, 12:28:28...	Jun 6, 2024, 12:28:28 PM
2.	airflow_monitoring	17	17	0	2	6	23	Jun 6, 2024, 12:09:22...	Jun 6, 2024, 2:40:00 PM

1 - 2 / 2

DAG Run Statistics

	Dag Id	Run Id	Run State	Run Duration (Sec)	Run Start Time (UTC)	Run End Time (UTC)	Total Tasks	Min Task Duration (sec)	Median Task Duration (sec)	Max Task Duration (sec)
1.	airflow_monitoring	scheduled__2024-06-06T14:30:00+00:00	success	8	Jun 6, 2024, 2:40:00 PM	Jun 6, 2024, 2:40:08 PM	1	3	3	3
2.	airflow_monitoring	scheduled__2024-06-06T14:20:00+00:00	success	11	Jun 6, 2024, 2:30:00 PM	Jun 6, 2024, 2:30:11 PM	1	4	4	4
3.	airflow_monitoring	scheduled__2024-06-06T14:10:00+00:00	success	8	Jun 6, 2024, 2:20:00 PM	Jun 6, 2024, 2:20:08 PM	1	3	3	3
4.	airflow_monitoring	scheduled__2024-06-06T14:00:00+00:00	success	5	Jun 6, 2024, 2:10:00 PM	Jun 6, 2024, 2:10:05 PM	1	3	3	3
5.	airflow_monitoring	scheduled__2024-06-06T13:50:00+00:00	success	10	Jun 6, 2024, 2:00:00 PM	Jun 6, 2024, 2:00:10 PM	1	3	3	3

1 - 18 / 18

It also provides a detailed examination on a task-by-task basis.

Run Task Statistics

	Dag Id	Run Id	Run State	Task Id	Task State	Task Operator	Task Duration (sec)	Task Start Time (UTC)	Task End Time (UTC)
1.	airflow_monitoring	scheduled__2024-06-06T11:50:00+00:00	success	echo	success	BashOperator	3	Jun 6, 2024, 12:09:41 PM	Jun 6, 2024, 12:09:44 PM
2.	airflow_monitoring	scheduled__2024-06-06T12:00:00+00:00	success	echo	success	BashOperator	1	Jun 6, 2024, 12:10:02 PM	Jun 6, 2024, 12:10:03 PM
3.	airflow_monitoring	scheduled__2024-06-06T12:10:00+00:00	success	echo	success	BashOperator	1	Jun 6, 2024, 12:20:01 PM	Jun 6, 2024, 12:20:02 PM
4.	trigger_rule_sample	manual__2024-06-06T12:28:27.043901+00:00	success	group1.task1_failed	failed	BashOperator	4	Jun 6, 2024, 12:28:33 PM	Jun 6, 2024, 12:28:37 PM
5.	trigger_rule_sample	manual__2024-06-06T12:28:27.043901+00:00	success	group1.task2_success	success	BashOperator	5	Jun 6, 2024, 12:28:35 PM	Jun 6, 2024, 12:28:40 PM
6.	trigger_rule_sample	manual__2024-06-	success	group1.task3_failed	failed	BashOperator	5	Jun 6, 2024, 12:28:35 PM	Jun 6, 2024, 12:28:40 PM

1 - 24 / 24

Example Looker Studio Screen